

Singly Linked List Operations

Subject: Data Structures Topic: Linked Lists

Overview

Complete guide to singly linked list insertion, deletion, and traversal

Node structure

- Each singly linked node contains a value and a reference to the next node; the final node points to null.
- A head reference identifies the first node. Maintaining a tail reference makes append $O(1)$.
- Unlike arrays, nodes need not be adjacent in memory and do not support constant-time indexed access.

Insertion and deletion

- Insert at the head by linking the new node to the old head, then updating head; this is $O(1)$.
- Insert after a known node by changing two links; finding a position first still costs $O(n)$.
- Deleting after a known predecessor is $O(1)$; deleting by value requires a traversal and predecessor tracking.

Traversal and complexity

- Walk from head by following next until null; traversal and search are $O(n)$.
- A list uses $O(n)$ node storage plus one or more link fields per node.
- A singly linked list cannot move backward without another traversal; a doubly linked list stores a previous link at extra memory cost.

Correctness checks

- Handle empty-list insertion and deletion, single-node deletion, head changes, and tail changes.
- Never dereference a null link; detach removed nodes when the language/runtime requires cleanup.
- Use slow and fast pointers to detect cycles in $O(n)$ time and $O(1)$ extra space.